

Metodologia, narzędzia i stack technologiczny — MoodFlow

Krótkie zestawienie sposobu pracy zespołu, używanego oprzyrządowania i technologii wybranych do realizacji projektu.

1. Metodologia pracy

Model pracy

- **Scrum-lite + Kanban** — hybryda zwinna dostosowana do dwuosobowego zespołu studenckiego.
- **Iteracje 2-tygodniowe** — zamknięty zakres funkcjonalności na sprint, krótki planning + retro.
- **Asynchroniczna komunikacja** — codzienne updatey statusu zastępują formalne stand-upy.
- **Praca w parze przy modułach krytycznych** — auth, multi-tenant, anonimizacja, scoring.

Tablica zadań (Kanban)

- **Backlog** — pomysły i zadania bez przypisanej iteracji.
- **To do** — zadania zaplanowane na bieżący sprint.
- **In progress** — zadania w trakcie realizacji (limit WIP = 2 / osobę).
- **Done** — zadania zakończone i zatwierdzone do scalenia.

Priorytetyzacja zakresu — MoSCoW

Każda funkcjonalność z PRD ma przypisany jeden z czterech priorytetów: **Must Have** (zakres MVP, krytyczne dla obrony), **Should Have** (istotne, planowane do MVP), **Could Have** (jeśli zostanie czas), **Won't Have** (poza zakresem pracy). Klasyfikacja jest źródłem prawdy dla harmonogramu (rozdział 4) i decyzji o zakresie.

Definition of Done

- kod przeglądnięty w PR przez drugiego członka zespołu;
- ESLint i typecheck TypeScript nie zgłaszają błędów;
- endpoint backendowy pokryty testem klikalnym lub jednostkowym;
- frontend manualnie sprawdzony w obu aplikacjach (pracownik + admin);
- zmiana udokumentowana w README, komentarzu PR lub rozdziale pracy.

Kontrola wersji

- **Monorepozytorium Git** z trzema strefami: backend-api, client-frontend, admin-frontend.
- **Pull Requesty** — jedyna droga zmian na master; obowiązkowy review.
- **Conventional Commits (PL)** — np. *fix(client-frontend): wymuś jasne tokeny tekstu na ekranach auth*.
- **Migracje TypeORM** — każda zmiana schematu posiada plik forward + revert.

2. Stack technologiczny

Aplikacja webowa trójwarstwowa: dwie aplikacje frontendowe (pracownik / HR + admin) komunikują się z centralnym API. Wszystkie warstwy w TypeScript.

Backend (backend-api)

Element	Technologia	Uzasadnienie
Framework	NestJS 11	modularna struktura, DI, podział na kontrolery/serwisy
Język	TypeScript 5.9	statyczne typowanie DTO i modeli domenowych
ORM	TypeORM 0.3	migracje + repozytoria, dobra integracja z NestJS
Baza danych	PostgreSQL 16	relacyjna, wielotabelowa: użytkownicy + ankiety + wyniki
Autentykacja	JWT + Passport	access + refresh, RBAC pracownik/HR/admin
Hashowanie haseł	bcrypt	standard branżowy
Walidacja DTO	class-validator	deklaratywne reguły na poziomie kontrolerów
Rate limiting	@nestjs/throttler	ochrona logowania przed brute-force
Nagłówki HTTP	helmet	podstawowa ochrona XSS / clickjacking
E-mail	Nodemailer	powiadomienia (zatwierdzenie konta, nowy test)

Frontend pracownika (client-frontend)

Element	Technologia	Uzasadnienie
Framework UI	React 19	komponentowy model UI
Build tool	Vite 8	szybki HMR, krótkie czasy buildu
Style	Tailwind CSS 3	utility-first, design system w klasach
State	Redux Toolkit 2	store sesji i meta-danych użytkownika
Routing	React Router 7	ochrona widoków przez strażników
Animacje	Framer Motion 12	subtelne przejścia
Ikony	Lucide React	spójna paleta line-art
PWA	vite-plugin-pwa	instalowalność + offline po pierwszym wejściu

Frontend administracyjny / HR (admin-frontend)

Element	Technologia	Uzasadnienie
Framework UI	Next.js 16 (App Router)	uporządkowana struktura panelu
Style	Tailwind CSS 4	ten sam design system co u pracownika
Wykresy	Recharts 3	trendy HR (linowe i słupkowe)
Eksport	jsPDF + html2canvas	raport HR do PDF po stronie klienta
Ciasteczka	js-cookie	zarządzanie sesją w panelu
Theming	next-themes	tryb jasny / ciemny
PWA	@ducanh2912/ next-pwa	instalowalność panelu HR

Infrastruktura i wdrożenie

Element	Technologia	Uzasadnienie
Konteneryzacja	Docker + Docker Compose	powtarzalne środowisko dev i prod
Środowisko prod	Railway	orkiestracja kontenerów + managed PostgreSQL
VCS	Git + GitHub	monorepozytorium, ochrona master, PR-y
CI (planowane)	GitHub Actions	lint + typecheck + testy klikalne przy każdym PR

3. Wykorzystywane narzędzia

Zestaw narzędzi został dobrany tak, by zminimalizować tarcia pomiędzy pracą developerską a pracą dokumentacyjną. Większość pozycji jest darmowa w wariacie studenckim lub open-source.

Narzędzia developerskie

Kategoria	Narzędzie	Zastosowanie
IDE	Visual Studio Code, Cursor	edycja kodu + integracja z AI
Środowisko	Docker Desktop	baza i usługi lokalnie
Klient bazy danych	TablePlus, pgAdmin	przeglądanie schematu, ad-hoc query
Klient HTTP	Postman, REST Client (VS Code)	testowanie endpointów API
Lint / formatter	ESLint, Prettier	konwencje kodu
Typecheck	TypeScript Compiler	weryfikacja typów w obu warstwach
Migracje DB	TypeORM CLI	wersjonowanie schematu
Build	Vite, Next.js build, nest build	produkcyjne paczki
Testy	Jest (backend), Vitest (front)	testy jednostkowe i integracyjne

Narzędzia współpracy i dokumentacji

Kategoria	Narzędzie	Zastosowanie
Kontrola wersji	GitHub	PR-y, code review, ochrona master
Tablica zadań	GitHub Projects (Kanban)	śledzenie sprintów i statusu zadań
Komunikacja	Discord, Slack	krótkie ustalenia i wymiana plików
Spotkania	Google Meet, MS Teams	konsultacje z promotorem, sync zespołu
Diagramy	draw.io, Mermaid, PlantUML	ERD, przypadki użycia, sekwencje
Design UI	Figma, Pencil	makiety i prototypy, biblioteka komponentów
Pisanie pracy	Microsoft Word, python-docx	rozdziały + generowanie tabel
Notatki / wiedza	Notion, Obsidian	notatki badawcze, ADR
Wsparcie AI	Claude Code, ChatGPT	review kodu, generowanie fragmentów docs
Hasła i sekrety	1Password	bezpieczne przechowywanie sekretów rozwojowych